🚀 Desarrollo y Operaciones con agentes de IA

IA como Copiloto · Parte 8 de 9

👤 Sergi Garcia Barea

CC BY-SA 4.0

🎯 El cierre del círculo: dev + ops

Hasta aquí: código, tests, skills y herramientas conectadas. Falta la otra mitad del ciclo de vida: **configurar, desplegar y operar**. Los LLM generan eso igual de bien que funciones.

💻 Dev (partes 1–5)

Código, tests, refactor, debugging, documentación. El producto en sí.

⚙️ Ops (esta parte)

Contenedores, pipelines, configuraciones, scripts de despliegue, monitorización.

🧩 Todo es texto con reglas

Un Dockerfile o un YAML de CI son tan «generables» como una clase. Si se pueden describir, se pueden generar y revisar.

💡 La idea de fondo: **la infraestructura también va en el repo, en ficheros** — el agente vive exactamente ahí.

🐳 Docker y compose generados con criterio

📝 PROMPT

```
"Dockeriza la app Flask (es la parte 4):

- Multi-stage: builder con dependencias
  pinned, runtime con python:slim
- Usuario no root
- .dockerignore: venv, tests, .env, git
- Compose: app + postgres + volúmenes
- Healthcheck en /health, con dependencias
  ordenadas (BD antes que la app)
- Variables con .env y defaults de
  desarrollo

Que compose up quede funcionando en local."
```

OpenCode · Docker · IaC

💡 QUÉ AÑADE DE FÁBRICA

- ✅ Multi-stage y slim → imagen pequeña, no la de mil tutoriales
- ✅ Usuario no root y .env de solo ejemplo: buenas prácticas por defecto
- ✅ Healthcheck y orden de arranque: los fallos de "a veces no conecta" desaparecen
- ✅ Si algo no arranca, el agente lee los logs del contenedor y corrige
- ✅ El Dockerfile queda en el repo: como código, se revisa

🔊

🔧 CI: pipeline de GitHub Actions

📝 PROMPT

```
"Crea .github/workflows/ci.yml:

- Dispara en cada PR y push a main
- Job: Node 22, npm ci, lint,
  test y build
- Cachear node_modules
- Estado del job como comentario
  en el PR

Aclárate por qué cada paso
  está ahí. Vamos a mantenerlo
  el curso."
```

OpenCode · CI/CD · Pipelines

💡 PIPELINE EN EL REPO

- ✅ Cada PR se compila y testea sin que nadie se acuerde
- ✅ El agente puede consumir el log del job si falla y propones arreglo
- ✅ "Por qué cada paso" → el YAML deja de ser magia negra
- ✅ El siguiente nivel: CD al desplegar cuando el CI pasa
- ✅ Revisiones: el YAML también pasa por PR, como el código

🔊

📜 Scripts de despliegue y de operar

📝 PROMPT

```
"Genera scripts/:

- desplegar.sh: build, guardar imagen,
  subir a registry y reiniciar el
  contenedor remoto por SSH
- backup.sh: dump de la BD con fecha
  en el nombre y limpieza de copias
  > 30 días
- healthcheck.sh: comprueba /health
  y avisa con error no cero si falla

Ambos con --help, set -euo pipefail
y ejecutar en seco con --dry-run."
```

OpenCode · Scripts · Bash

💡 BESOS BÁSICOS QUE PIDE EL PROMPT

- ✅ **set -euo pipefail**: que pare, no que siga roto
- ✅ **--dry-run**: simular antes de ejecutar en el servidor
- ✅ **--help**: los scripts se autodocumentan
- ✅ Sin secretos en el código: todo por variables de entorno
- ✅ Los scripts se testean en el CI como si fueran código más

🔊

🩺 Operación: logs, métricas y esperado

💡 Endpoint /health

El agente puede añadirlo: BD conectada, migraciones al día, versión expuesta.

📊 Logs útiles

"Añade logs estructurados con request, estado y duración." Sin spam ni secretos.

🧪 Script de observabilidad

Un script que consulta /health y métricas y resume con una línea verde o roja por servicio.

📈 Qué mirar

"¿Qué deberíamos monitorizar en esta app y por qué?" — pregunta con criterio antes de instrumentar.

🚨 Detección de incidentes

Con el MCP de BD o de logs, el agente hace hipótesis, busca en datos y descarta causas.

🫰 Post-mortem

Con la línea temporal de Git y los logs, el agente redacta causas y acciones preventivas.

🎯 Monitorizar sin saber por qué es ruido. Pedirle "qué y por qué" al copiloto es el atajo del aprendizaje.

🔊

🔐 Auditoría de seguridad con el copiloto

📝 PROMPT

```
"Auditoría rápida del repo:

1. Dependencias desactualizadas
   o con avisos de seguridad
2. Secretos o claves comitadas
   por accidente (historial + .env)
3. Endpoints sin autentificar
   que deberían tenerla
4. Inyección SQL / XSS a mano
   en lo vuestro, no en librerías

Report con severidad y parche
   propuesto por hallazgo.
   No cambies código todavía."
```

OpenCode · Auditoría · Report

💡 CÓMO SE USA BIEN

- ✅ "No cambies código todavía" → report primero, parches aprobados después
- ✅ Es una primera pasada, no un pentest profesional
- ✅ Complementa (no sustituye) a herramientas dedicadas
- ✅ Cero coste: se puede ejecutar antes de cada entrega
- ✅ Los hallazgos son una tarea planificable como cualquier otra

🔊

⚖️ Uso responsable del copiloto

🧠 Aprender > copiar

Si el agente te soluciona un bug y no entendiste el porqué, has perdido el tiempo que creías ganar. Pide la explicación.

👀 Deberías poder defender tu código

Cada línea del repo puede preguntársete en una defensa o revisión. Si no la entiendes, no la aceptes.

🚫 No disimular

Atribuye el uso de IA como se pida en cada contexto, y nunca para disimular copias. Delegar no elimina la responsabilidad.

🔒 Secretos fuera

Ni API keys ni datos personales en prompts. Todo por variables de entorno y datos de prueba

⚖️ Licencias

El código generado es tuyo según las condiciones del servicio; cuida no contaminar el repo con fragmentos de licencias restrictivas.

📏 Límites de la capa gratis

Cuotas por día/hora: divide el trabajo en sesiones cortas, igual que ya hace Freebuff.

🔊

🧭 El flujo completo, de principio a fin

Fase	Qué hace el agente	Qué sigue haciendo el humano
Idea → plan	Explora, propone opciones y pasos	Decide y aprueba el plan
Implementación	Escribe código, tests y migración	Revisa diffs, dirige el alcance
Verificación	Ejecuta tests, prueba en navegador	Acepta o pide cambios
Integración	Rama, PR, CI	Revisión y merge
Operación	Docker, despliegue, healthchecks, reportes	Aprende y confirma en cada caso real

El copiloto acelera las casillas del medio. Las decisiones y la responsabilidad son tuyas en todo el recorrido.

🔊

🎓 Tu próximo proyecto, con copiloto

📋 1. Proyecto: repositorio + AGENTS.md

El repo desde el minuto 1 y el manual del proyecto creciendo con cada decisión.

🗺️ 2. Plan de features

Un PLAN.md con criterios de aceptación. Un paso por iteración, commit verificado.

🔌 3. Herramientas conectadas

BD en readonly y GitHub para issues/PRs. Menos es más.

🧪 4. Calidad bajo pedido

Tests en cada módulo, auditorías con el prompt de esta parte, sin despliegues a ciegas.

📑 5. Documentación viva

README, memoria y diagramas generados al ritmo del código, con lo ⚠️ marcado para revisar.

🧠 6. Aprende, no deleges

Pide explicaciones, defiende tu código y cuándo acabes, sabrás de qué está hecho. No solo que funciona.

🔊

✨ Resumen de la serie completa

1 · **Agentes de código** — qué son, gratis: OpenCode + Freebuff, y las alternativas: Claude Code, Codex, Antigravity

2 · **Plan** → **Hacer** — plan aprobado, pasos pequeños, verificación y commits limpios

3 · **AGENTS.md y skills** — el manual del proyecto y las recetas por tarea

4 · **Ejemplos de desarrollo** — código, BD, tests, refactor, debug, docs y configuración

5 · **Plugins y MCPs** — GitHub, BD readonly y navegador: el agente conectado, con permisos justos

6 · **Documentación como código** — Markdown, Mermaid, Excalidraw y ADRs versionados con el código

7 · **Infraestructura como código** — Docker Compose, Terraform, Containerlab y Ansible generados y revisados

8 · **Dev + Ops** — Docker, CI/CD, scripts de despliegue, observabilidad y uso responsable

9 · **OpenSpec** — el plan como contrato: specs, cambios y archivo dentro del repo

🎯 IA como copiloto de desarrollo y operaciones: **tú pilotas, la IA acelera** — y siempre revisa.

Siguiente: **OpenSpec** — las especificaciones como contrato entre tú y el agente. 📋

CC BY-SA 4.0 · Sergi Garcia Barea · Octubre 2026

🔊